

TRACE-HEP

Comprehensive Documentation - v0.1.1 - Toolkit for
Reading Annotated Collider-Event displays

What is TRACE-HEP?

- * A format-agnostic Python library for reconstructed-level collider event and vertex displays
 - * Polar (ϕ , $|\eta|$), beam-axis 2D, and interactive 3D event views
 - * A companion per-vertex / all-vertices family for pileup diagnostics
 - * Core package has zero ROOT/uproot dependency
 - * Optional loaders for Delphes, flat-ntuple, and calo-timing ROOT files
 - * Open source, MIT licensed

Where to find it

GitHub: github.com/wasikatcern/TRACE-HEP
TestPyPI: test.pypi.org/project/trace-hep
License: MIT

Installation: core package

```
python3 -m venv .venv  
source .venv/bin/activate
```

```
pip install --index-url https://test.pypi.org/simple/ \  
--extra-index-url https://pypi.org/simple/ \  
trace-hep
```

Core install pulls only numpy, matplotlib, and plotly.

Installation: optional extras

```
# ROOT-file loaders: Delphes, flat ntuple, calo-timing
pip install --index-url https://test.pypi.org/simple/ \
            --extra-index-url https://pypi.org/simple/ \
            "trace-hep[delphes]"

# + static image export (kaleido) and ATLAS style (mplhep)
pip install ... "trace-hep[all]"

# from source instead, editable (for developing tracehep itself)
git clone https://github.com/wasikatcern/TRACE-HEP.git
cd TRACE-HEP
pip install -e ".[dev]"
pytest -q
```

Verify the install

```
python3 -c "import tracehep as trace; print(trace.__version__)"  
# -> 0.1.1
```

```
trace-batch --help  
# -> usage: trace-batch [-h] --input FILE.root --indices N [N ...] ...
```

The data model: one hard-scatter event

```
        Jet(pt, eta, phi, mass=0, btag=False)
Track(pt, eta, phi, charge=0, d0=0, z0=0, x=0, y=0, z=0, time=None)
        Lepton(pt, eta, phi, flavor="muon"|"electron")
            Photon(pt, eta, phi)
            MissingET(pt, phi, eta=0)

Event(jets, tracks, muons, electrons, photons, met,
      run=None, event_number=None, label="")
```


The data model: one pileup scenario

```
Vertex(z, x=0, y=0, sum_pt2=0, is_hs=False, is_pu=False,  
       time=None, track_indices=[])  
    TruthVertex(z, x=0, y=0, is_hs=False)
```

```
VertexEvent(vertices, tracks, truth_vertices, mu=None, label="")
```

Nothing here knows about ROOT or any ntuple format -- build these by hand from your own analysis objects and every drawing function in the package works unchanged.

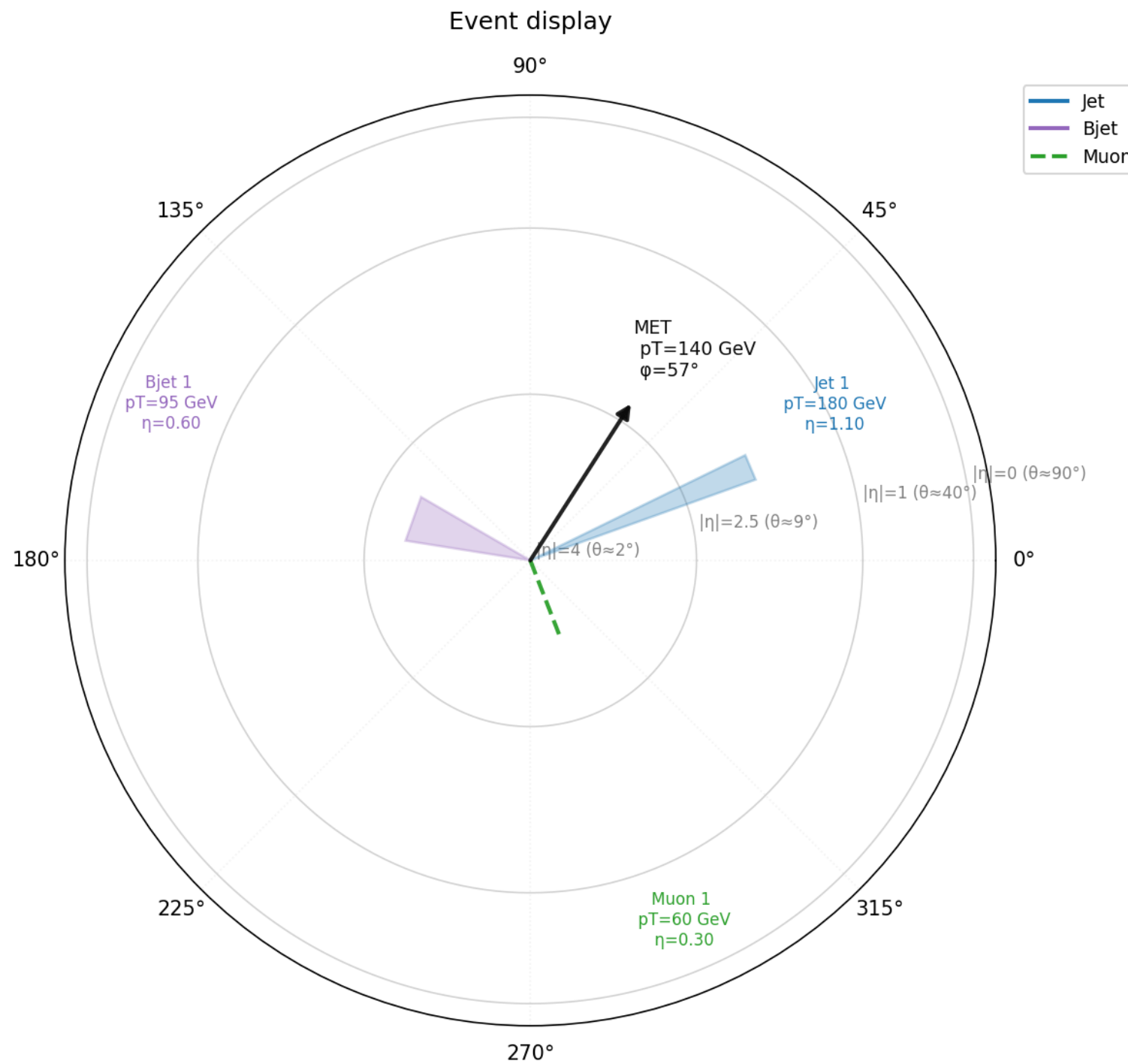
Quickstart: build an event by hand

```
import tracehep as trace

event = trace.Event(
    jets=[trace.Jet(pt=180, eta=1.1, phi=0.4),
          trace.Jet(pt=95, eta=-0.6, phi=2.8, btag=True)],
    muons=[trace.Lepton(pt=60, eta=0.3, phi=-1.2, flavor="muon")],
    met=trace.MissingET(pt=140, phi=1.0),
)

fig = trace.plot_event_polar(event)
fig.savefig("event_polar.png")
```

Output: the polar (phi, |eta|) view



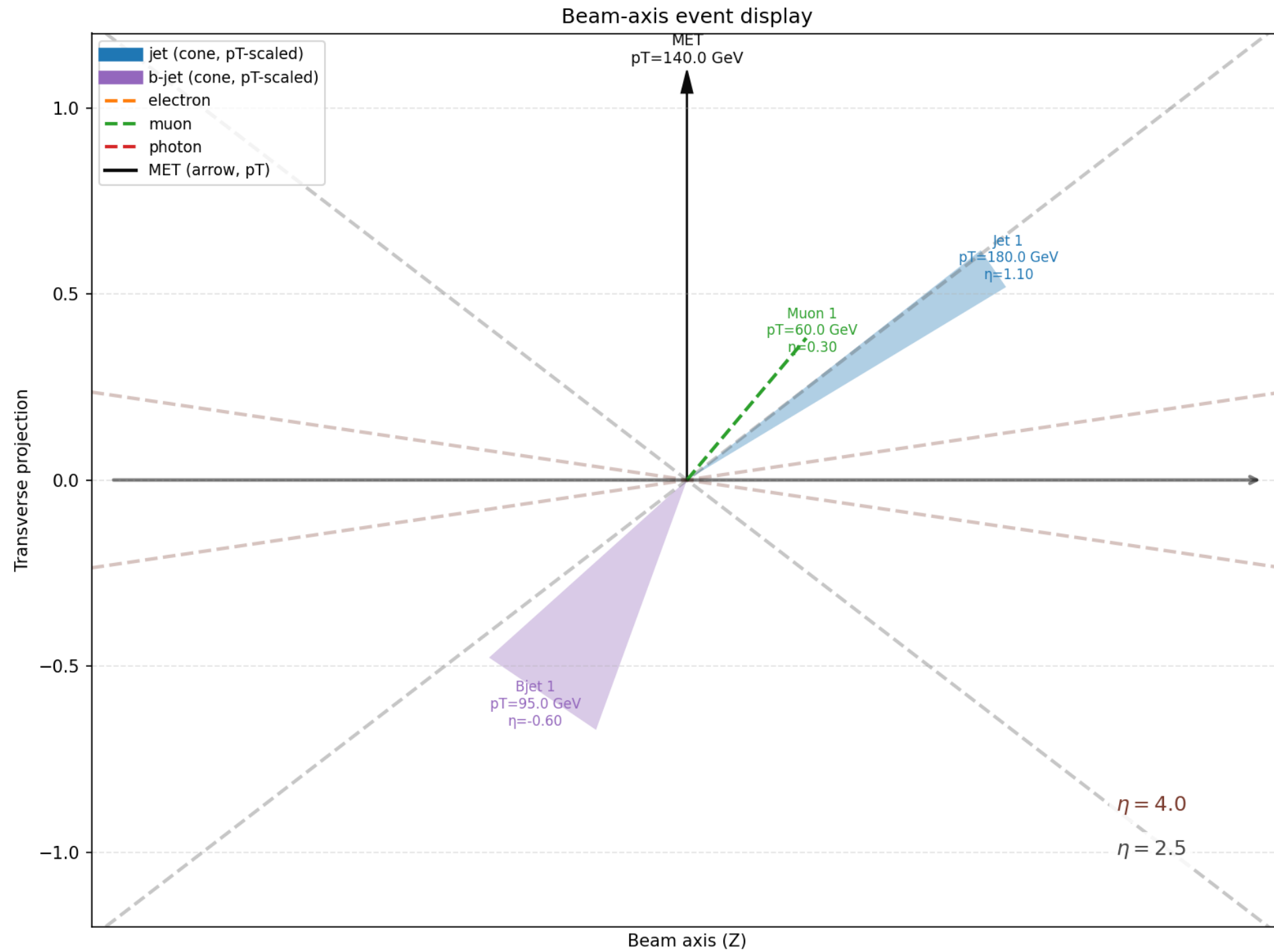
trace.plot_event_polar(event)

The beam-axis 2D projection

```
fig = trace.plot_event_beam2d(event)  
fig.savefig("event_beam2d.png")
```

Complementary side-on view: beam direction horizontal, transverse coordinate vertical.

Output: the beam-axis 2D view



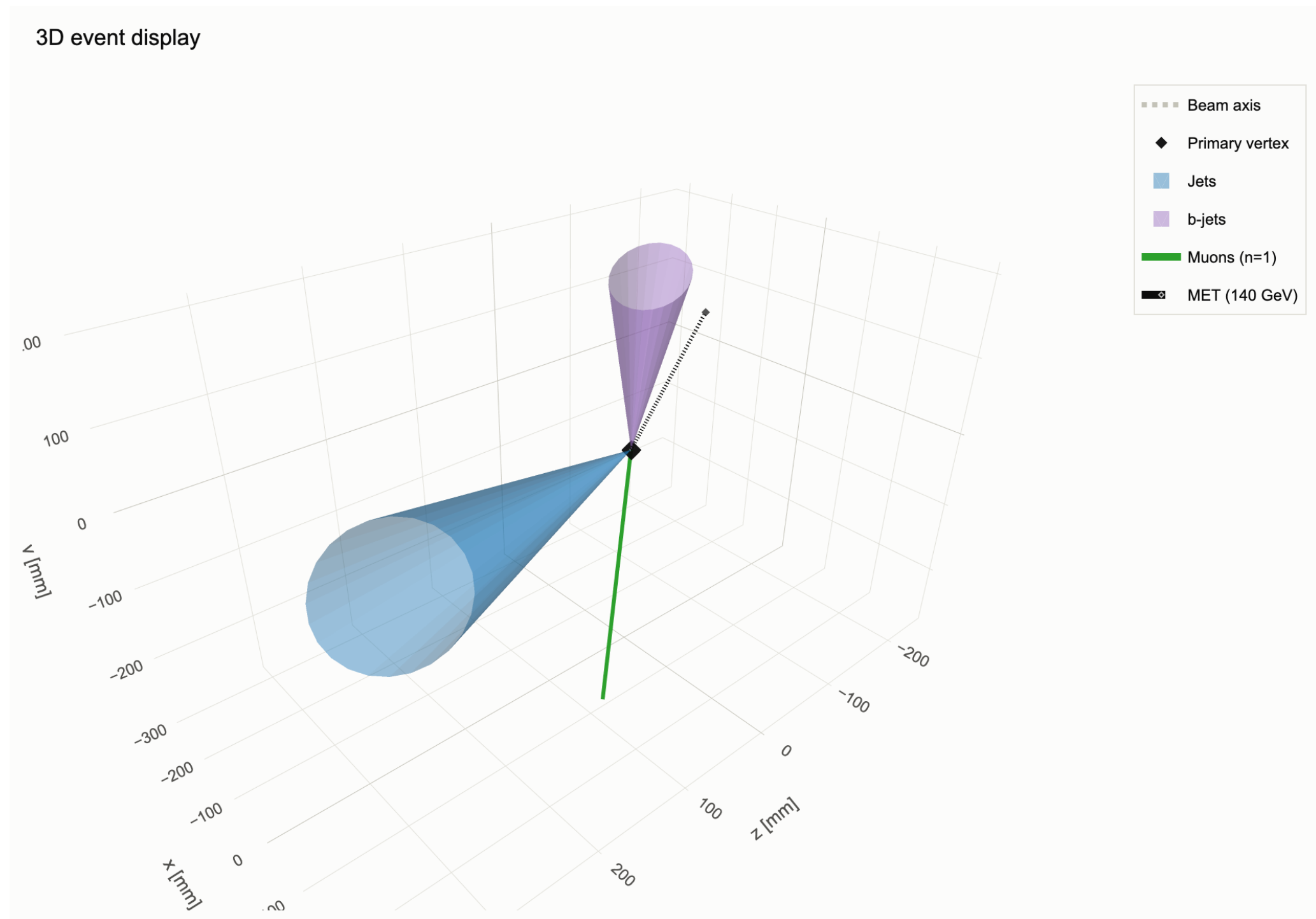
trace.plot_event_beam2d(event)

The interactive 3D view

```
fig3d = trace.plot_event_3d(event)
fig3d.write_html("event_3d.html")

# static snapshot (needs the kaleido extra)
fig3d.write_image("event_3d.png")
```

Output: the interactive 3D view



Static snapshot of the interactive HTML (drag to orbit, scroll to zoom)

Loading real events: Delphes ROOT files

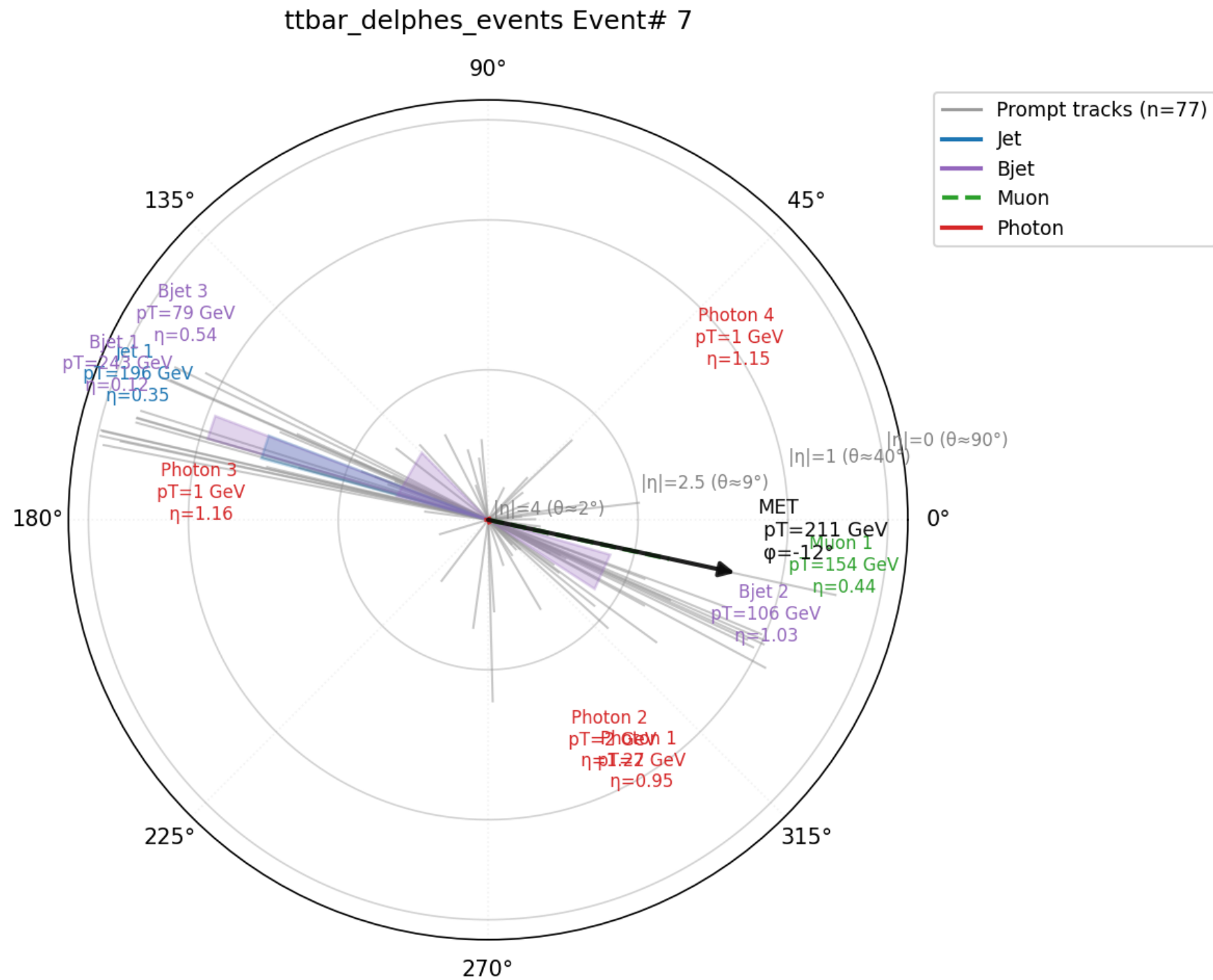
```
from tracehep.io.delphes import load_events

events = load_events(
    "ttbar_delphes_events.root",
    indices=[7],
    label="ttbar_delphes_events",
)
event = events[7]

fig = trace.plot_event_polar(event, show_tracks=True)
fig.savefig("event7_polar.png")
```

Needs the [delphes] extra. Works unchanged on any Delphes sample, any physics process.

Output: a real ttbar Delphes event



Event #7: 4 jets, 77 tracks, 1 muon, 4 photons, MET = 211 GeV

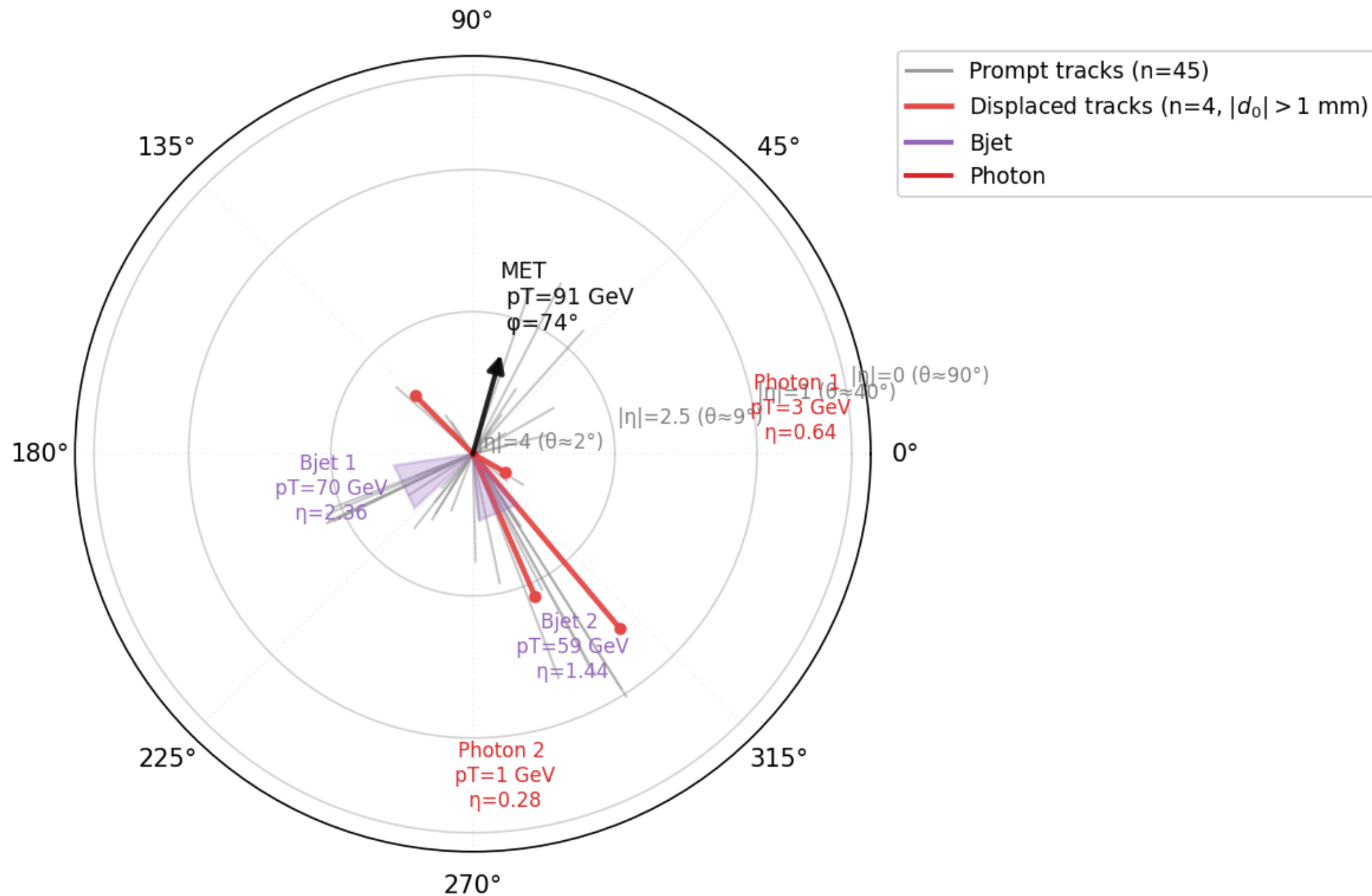
Highlighting displaced tracks (d0)

```
fig = trace.plot_event_polar(  
    event, show_tracks=True, d0_displaced_mm=1.0,  
    )  
fig.savefig("event_displaced_polar.png")  
  
fig3d = trace.plot_event_3d(event, d0_displaced_mm=1.0)  
fig3d.write_html("event_displaced_3d.html")
```

Tracks with $|d_0|$ above the threshold are coloured distinctly from prompt tracks.

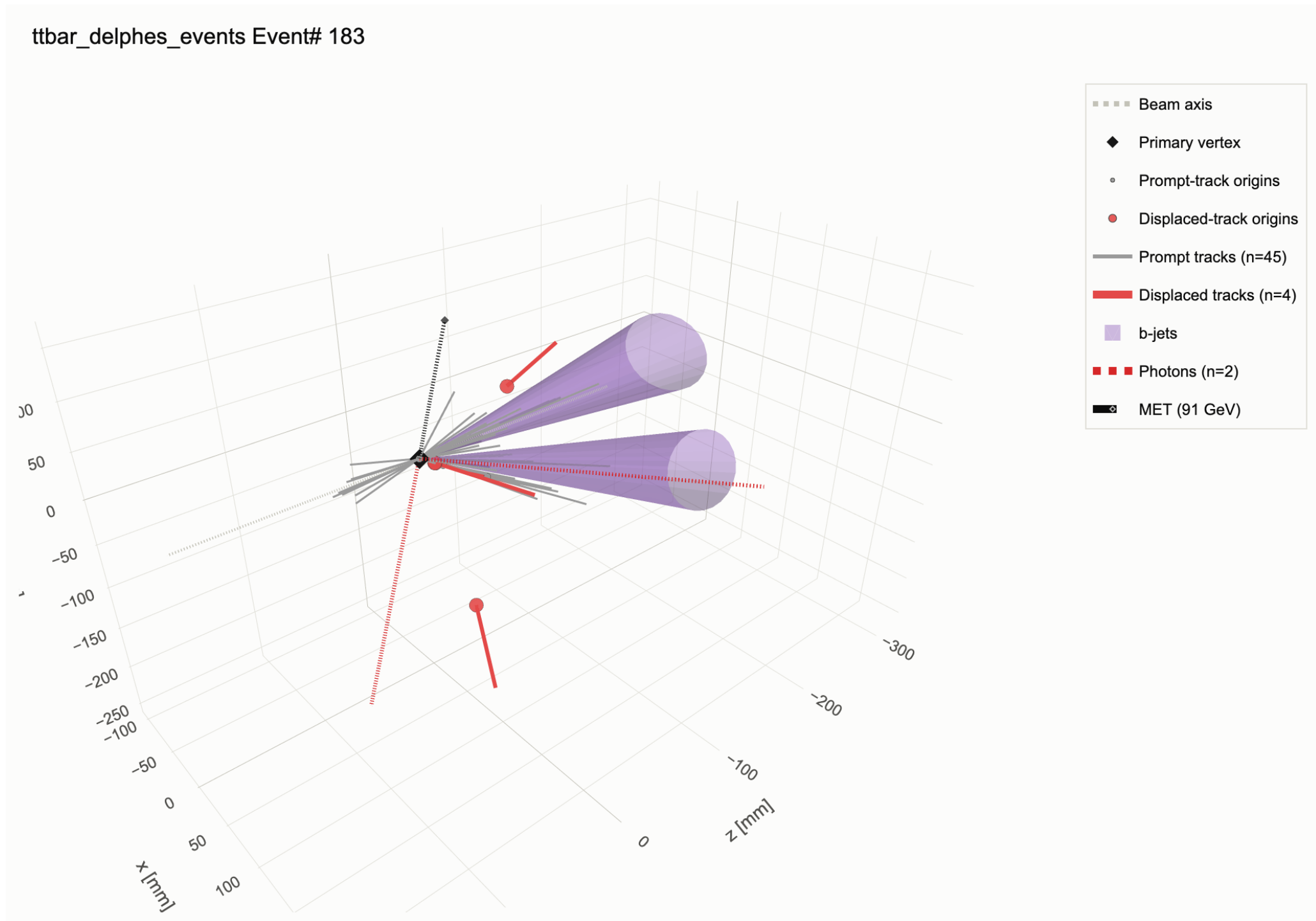
Output: displaced tracks (polar view)

ttbar_delphes_events Event# 183



4 of 45 tracks flagged as displaced ($|d_0| > 1$ mm), picked out in red

Output: displaced tracks (3D view)



Same event -- displaced-track origins marked distinctly in 3D

Vertex displays: loading a pileup ntuple

```
from tracehep.io.calotiming import load_vertex_event

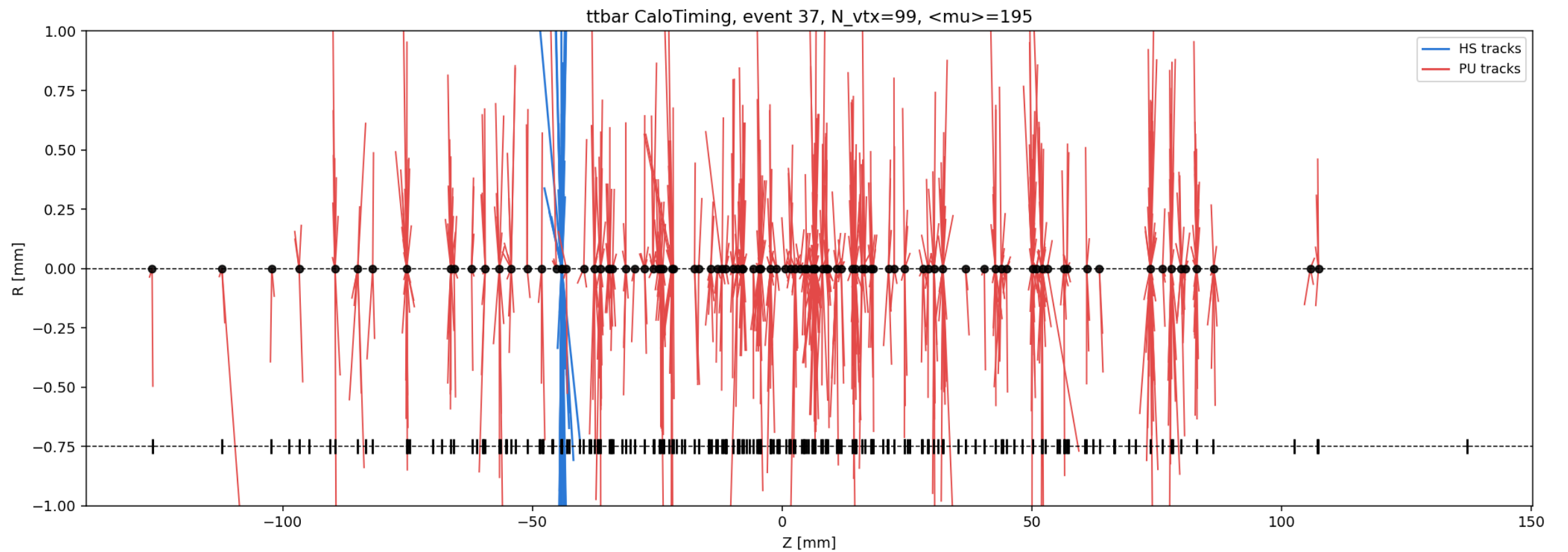
vtx_event = load_vertex_event(
    "calo_timing_ntuple.root",
    event_index=37,
)
print(len(vtx_event.vertices), "vertices")
print(vtx_event.mu, "<mu>")
# -> 99 vertices
# -> 195.0 <mu>
```

The z-R vertex display, three styles

```
from tracehep import plot_vertices_zr  
  
plot_vertices_zr(vtx_event, style="plain").savefig("plain.png")  
plot_vertices_zr(vtx_event, style="styled").savefig("styled.png")  
plot_vertices_zr(vtx_event, style="time_colored").savefig("time.png")
```

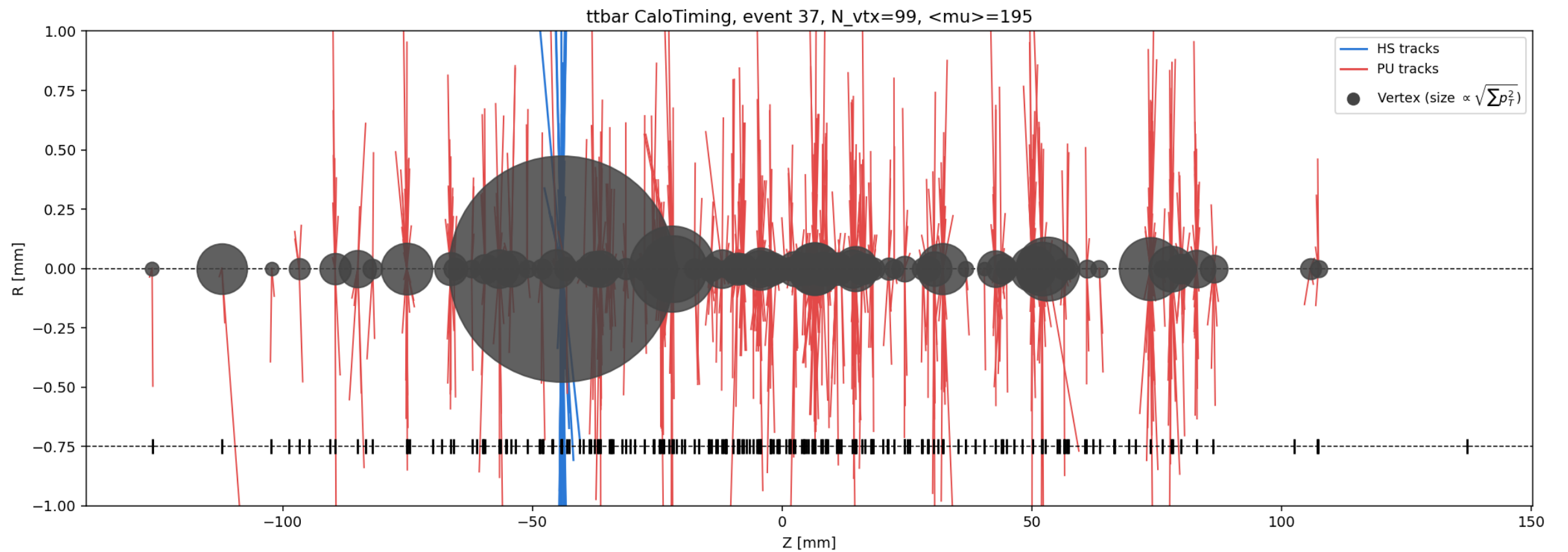
style is one of "plain", "styled", or "time_colored".

Output: style="plain"



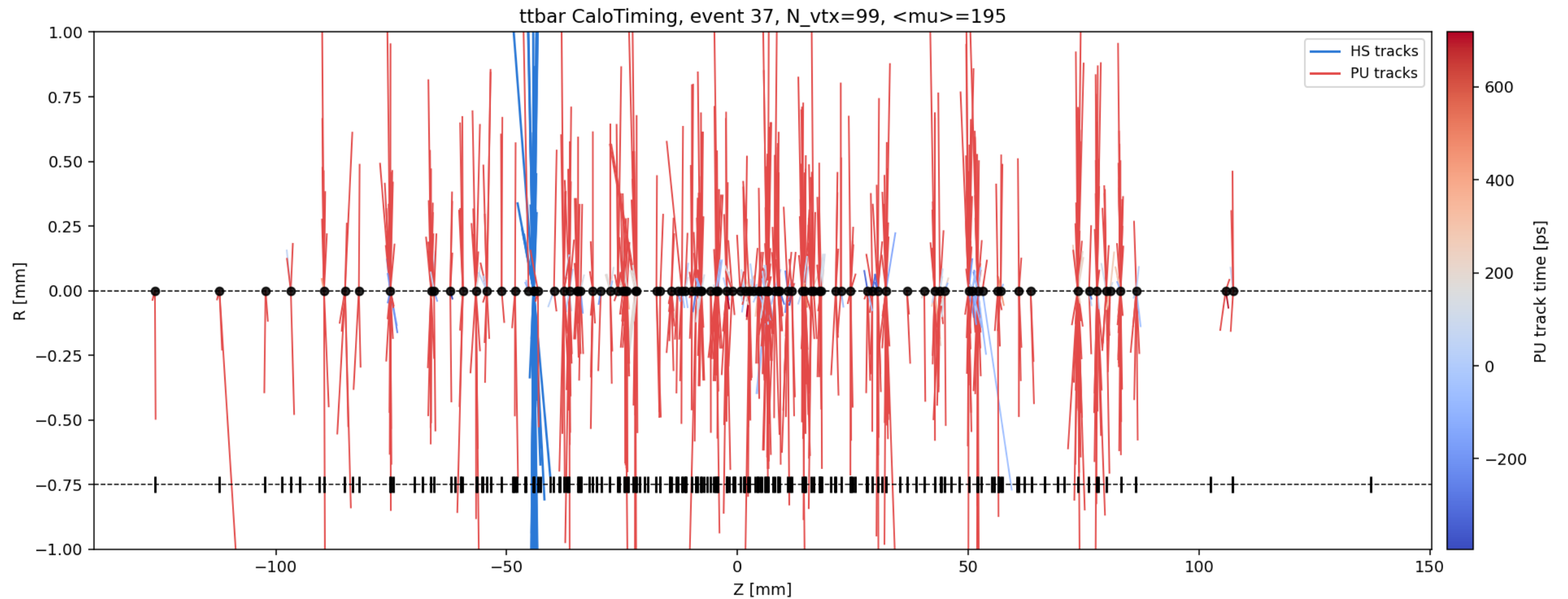
Flat HS (blue) / PU (red) track colouring, fixed vertex marker size

Output: style="styled"



Vertex marker size proportional to $\sqrt{\sum p_T^2}$; <mu> reported in the title

Output: style="time_colored"



PU tracks coloured by reconstructed track time (needs per-track timing)

Zooming into one vertex

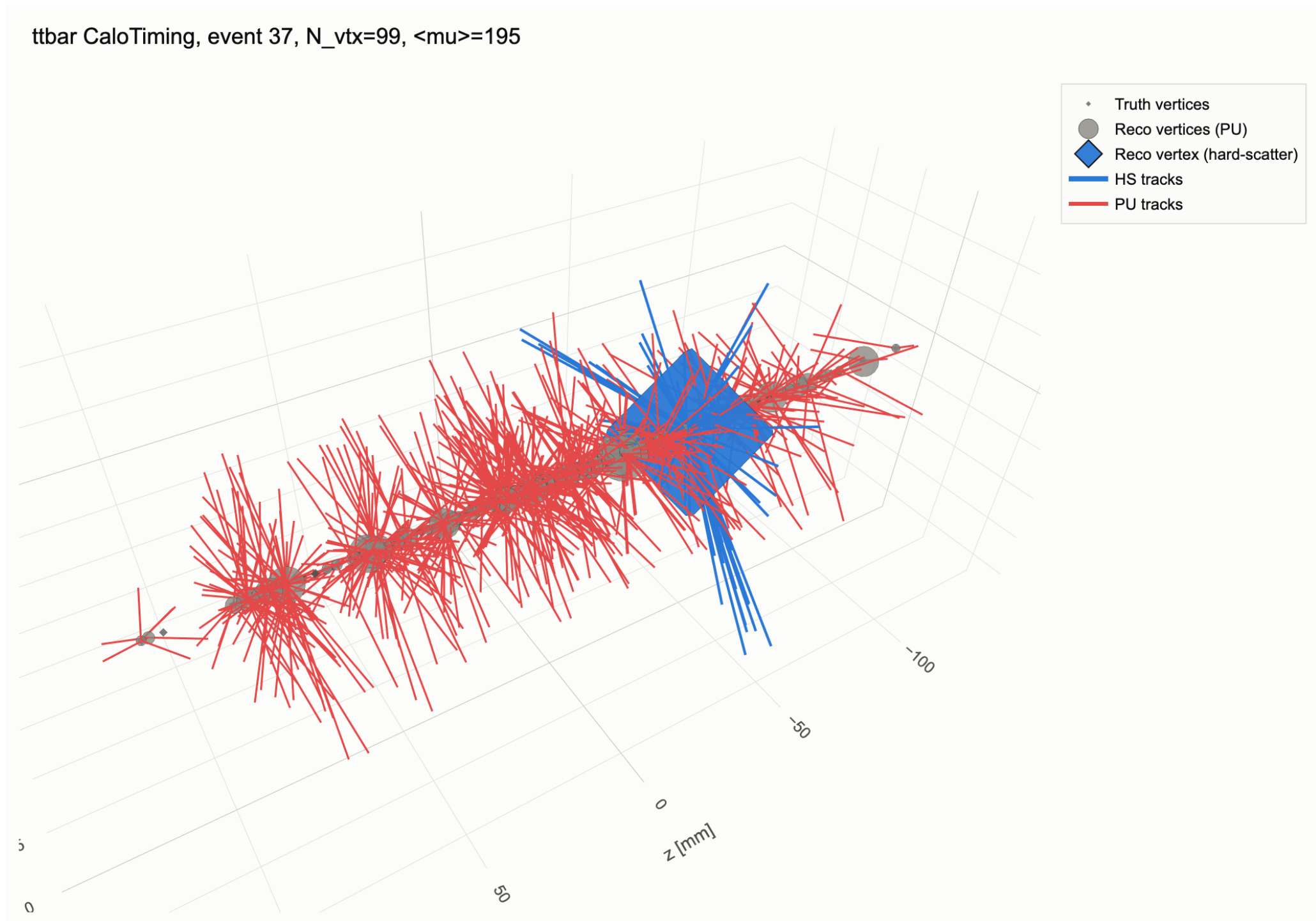
```
plot_vertices_zr(  
    vtx_event, style="styled",  
    zoom_center=-44.2, zoom_range_mm=5.0,  
    )
```

Same function -- pass zoom_center/zoom_range_mm to reproduce a single-vertex zoomed view.

Interactive 3D vertex display

```
from tracehep import plot_vertices_3d  
  
fig = plot_vertices_3d(vtx_event)  
fig.write_html("vertices_3d.html")
```

Output: the interactive 3D vertex view



Every reconstructed and truth vertex at its real (x, y, z) position

One vertex, with its associated jets

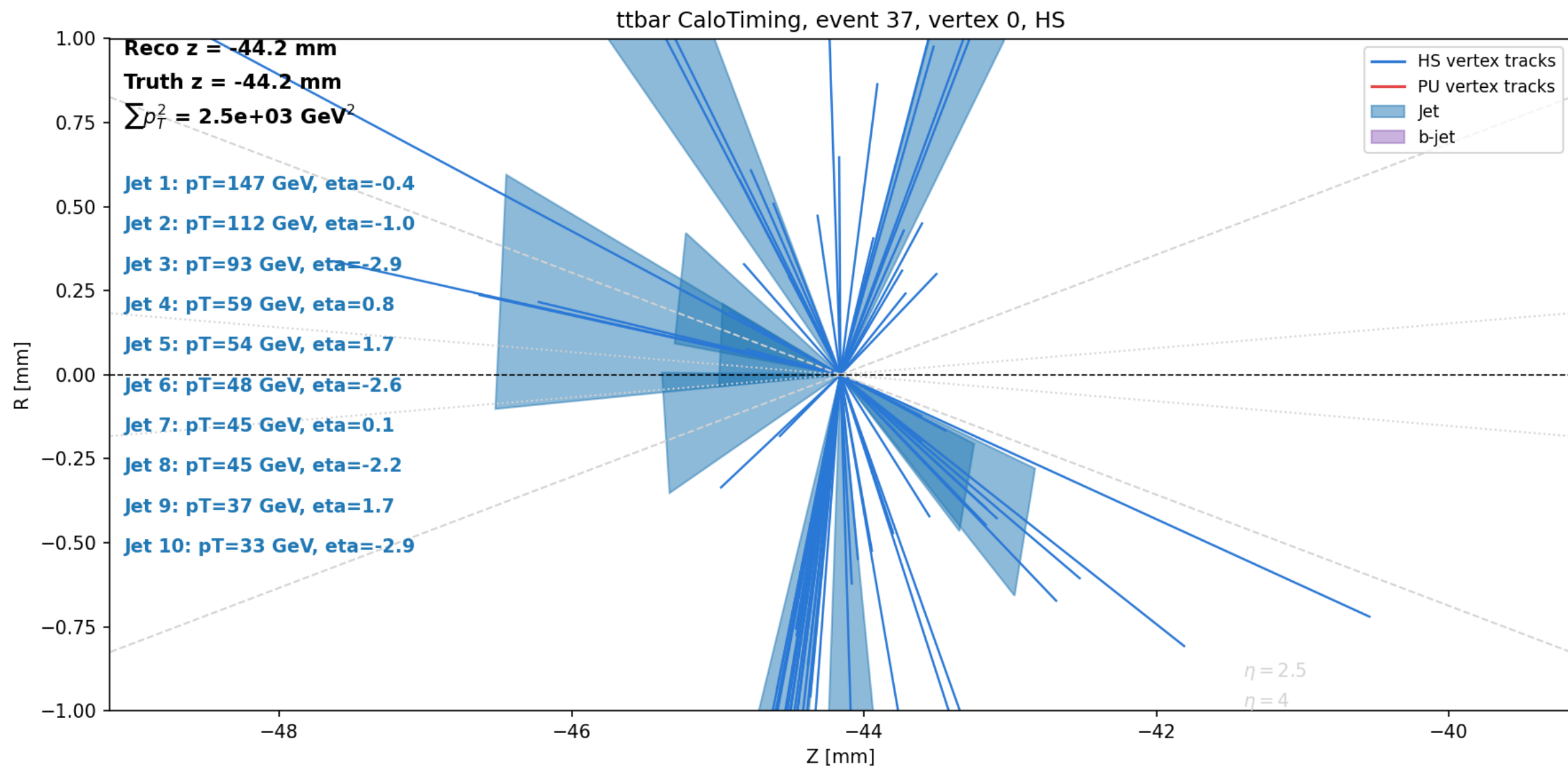
```
from tracehep import plot_vertex_detail
from tracehep.io.calotiming import match_jets_to_vertex

vtx = vtx_event.vertices[0]
jets = match_jets_to_vertex(
    "calo_timing_ntuple.root", event_index=37, vtx_z=vtx.z,
)

fig = plot_vertex_detail(vtx_event, vtx_index=0, jets=jets)
fig.savefig("vertex0_detail.png")
```

match_jets_to_vertex associates jets by Rpt (track-pT-fraction) matching -- bring your own jets for a different scheme.

Output: single-vertex detail with jets



Vertex 0 (HS): 10 jets matched, $\text{sum}(p_T^2) = 2.5e3 \text{ GeV}^2$ -- matches the original analysis exactly

Batch processing many events at once

```
from tracehep.io.delphes import load_events
from tracehep.batch import run_event_batch

events = load_events(path, indices=range(20))
run_event_batch(
    events, "out/", which=("polar", "beam2d"),
    polar_kwargs={"show_tracks": True},
)
# -> out/event0_polar.png, out/event0_beam2d.png, ...
```

Batch processing vertex scenarios

```
from tracehep.io.calotiming import load_vertex_event
from tracehep.batch import run_vertex_batch

vtx_events = {i: load_vertex_event(path, i) for i in range(10)}
run_vertex_batch(vtx_events, "out/", styles=("plain", "styled"))
# -> out/vertices0_plain.png, out/vertices0_styled.png, ...
```


The trace-batch command-line tool

```
trace-batch \  
--input ttbar_delphes_events.root \  
--indices 0 1 2 \  
--outdir out/ \  
--which polar beam2d \  
--show-tracks \  
--label my_sample
```

No Python required -- useful for quick batch regeneration from a shell script.

Function reference: event displays

```
plot_event_polar(event, *, ax=None, eta_max=4.0, pt_scale=0.003,  
                  eta_guides=(0,1,2.5,4.0), show_tracks=False,  
                  track_pt_scale=0.15, d0_displaced_mm=None, title=None)
```

```
    plot_event_beam2d(event, *, ax=None,  
eta_guides=(2.5,4.0), angle_scale=4.0, title=None)
```

```
plot_event_3d(event, *, d0_displaced_mm=None,  
                jet_half_angle_deg=9.0, title=None)
```

Function reference: vertex displays

```
plot_vertices_zr(vertex_event, *, style="plain",  
                 zoom_center=None, zoom_range_mm=None,  
                 ax=None, title=None)
```

```
plot_vertex_detail(vertex_event, vtx_index, jets=(),  
                  *, zoom_range_mm=5.0, ax=None, title=None)
```

```
plot_vertices_3d(vertex_event, *, title=None)
```

```
run_event_batch(events, outdir, *, which=(...),  
                polar_kwargs=None, beam2d_kwargs=None)  
run_vertex_batch(vertex_events, outdir, *, styles=(...))
```

Loader reference (tracehep.io)

```
delphes.load_event(path, index, with_tracks=True, label="")  
delphes.load_events(path, indices, with_tracks=True, label="")
```

```
flat_ntuple.load_event_by_run_event(path, run, event,  
                                     tree_name="Ntuple", label="")  
flat_ntuple.load_events_by_run_event(path, pairs, ...)
```

```
calotiming.load_vertex_event(path, event_index,  
                              tree_name="ntuple", label="")  
calotiming.match_jets_to_vertex(path, event_index, vtx_z,  
                                *, jet_pt_min=30.0, rpt_min=0.02, sig_cut=3.0)
```

All four require: `pip install "trace-hep[delphes]"`

Troubleshooting

- * ImportError: uproot required
-> pip install "trace-hep[delphes]"
- * ModuleNotFoundError: No module named tracehep (right after install)
 - > from TestPyPI you need BOTH --index-url and --extra-index-url (see Installation slides)
- * KeyError from flat_ntuple loader
-> the (run, event) pair is not present in that file
- * A track's 3D origin looks wrong from a calo-timing ntuple
 - > that format has no per-track (x, y, z); origin defaults to the owning vertex's (x, y) plus the track's own z0

Versioning and updates

- * Published package versions are immutable

-> updating requires bumping the version and re-uploading

- * Typical update loop:

- edit code in src/tracehep/

- add/update a test

- bump version in pyproject.toml and __init__.py

- python3 -m build

- twine upload --repository-url <https://test.pypi.org/legacy/> dist/*

Get involved

Source: `github.com/wasikatcern/TRACE-HEP`
Install: `pip install trace-hep` (TestPyPI now; PyPI planned)
License: MIT
Tests: `pytest -q` (21 passing at v0.1.1)
Contact: `wasikul.islam@cern.ch`

Citing: accompanying paper citation to be added once posted.